

模块 8：构建你自己的编程代理

模块概述

在前面的模块中，我们深入分析了 Cursor、Copilot、Claude Code 等成熟产品的架构与设计决策。现在，是时候将所学付诸实践了。

本模块将带你从零开始，一步步构建一个完整的编程代理（coding agent）。我们从最基础的 API 调用开始，逐步加入工具调用（tool use）、代码检索（grounding）、反馈循环（feedback loop）和规划机制（planning），最终构建出一个具备实用能力的编程代理。

你将学到： - 如何使用 Anthropic SDK 进行 tool use 调用 - 如何用 100 行代码实现一个最小可用的编程代理 - 如何通过 grounding 让代理理解大型代码库 - 如何通过测试驱动的反馈循环提高代理的正确率 - 如何加入规划机制处理复杂任务 - 企业环境下的安全性、隐私和成本控制

8.1 从 API 开始

在构建代理之前，我们需要掌握底层的 API 交互。本节以 Anthropic 的 Claude API 为例，介绍 tool use、streaming 和多轮对话管理。

8.1.1 环境搭建

Python：

```
pip install anthropic
```

TypeScript：

```
npm install @anthropic-ai/sdk
```

设置 API Key：

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

8.1.2 基础 API 调用

最简单的 API 调用——发送一条消息，获取回复：

Python:

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "用一句话解释什么是编程代理。"}
    ]
)

print(response.content[0].text)
```

TypeScript:

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

const response = await client.messages.create({
  model: "claude-sonnet-4-20250514",
  max_tokens: 1024,
  messages: [
    { role: "user", content: "用一句话解释什么是编程代理。" }
  ],
});

console.log(response.content[0].text);
```

8.1.3 添加 Tool Use

Tool use 是构建代理的核心能力。我们定义工具的 schema，模型决定何时调用哪个工具，我们执行工具并将结果返回给模型。

```
import anthropic
import json

client = anthropic.Anthropic()
```

```

# 定义工具 schema
tools = [
    {
        "name": "read_file",
        "description": "读取指定路径的文件内容",
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {
                    "type": "string",
                    "description": "文件路径"
                }
            },
            "required": ["path"]
        }
    },
    {
        "name": "list_directory",
        "description": "列出目录中的文件和子目录",
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {
                    "type": "string",
                    "description": "目录路径"
                }
            },
            "required": ["path"]
        }
    }
]

# 发送带工具定义的请求
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    tools=tools,
    messages=[
        {"role": "user", "content": "请读取 main.py 文件的内容"}
    ]
)

# 检查模型是否请求了工具调用
for block in response.content:
    if block.type == "tool_use":
        print(f"模型请求调用工具: {block.name}")
        print(f"参数: {json.dumps(block.input, ensure_ascii=False)}")
        print(f"工具调用 ID: {block.id}")

```

```
elif block.type == "text":
    print(f"模型文本回复: {block.text}")
```

8.1.4 处理工具调用结果

当模型返回 `tool_use` block 时，我们需要执行对应的工具，然后将结果以 `tool_result` 的形式返回：

```
import os

def execute_tool(name: str, input_data: dict) -> str:
    """执行工具并返回结果"""
    if name == "read_file":
        path = input_data["path"]
        if not os.path.exists(path):
            return f"错误: 文件 {path} 不存在"
        with open(path, "r") as f:
            return f.read()
    elif name == "list_directory":
        path = input_data.get("path", ".")
        if not os.path.isdir(path):
            return f"错误: {path} 不是一个目录"
        entries = os.listdir(path)
        return "\n".join(entries)
    else:
        return f"错误: 未知工具 {name}"

# 完整的工具调用往返
messages = [
    {"role": "user", "content": "请读取 main.py 文件的内容"}
]

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    tools=tools,
    messages=messages,
)

# 将 assistant 的回复加入消息历史
messages.append({"role": "assistant", "content": response.content})

# 处理工具调用
tool_results = []
for block in response.content:
    if block.type == "tool_use":
        result = execute_tool(block.name, block.input)
        tool_results.append({
```

```

        "type": "tool_result",
        "tool_use_id": block.id,
        "content": result
    })

# 将工具结果发回模型
if tool_results:
    messages.append({"role": "user", "content": tool_results})

# 模型根据工具结果生成最终回复
final_response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    tools=tools,
    messages=messages,
)
print(final_response.content[0].text)

```

8.1.5 Streaming 响应

对于交互式应用，streaming 能显著提升用户体验：

```

# Streaming 方式获取响应
with client.messages.stream(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "解释 Python 的 GIL 是什么"}
    ]
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)

print() # 换行

```

Streaming 模式下处理 tool use 需要在流结束后获取完整消息：

```

with client.messages.stream(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    tools=tools,
    messages=[
        {"role": "user", "content": "读取 main.py"}
    ]
) as stream:
    # 可以实时打印文本部分
    for text in stream.text_stream:
        print(text, end="", flush=True)

```

```
# 流结束后获取完整响应，处理工具调用
response = stream.get_final_message()
for block in response.content:
    if block.type == "tool_use":
        print(f"\n[调用工具: {block.name}]" )
```

8.1.6 多轮对话管理

编程代理的核心是持续的多轮对话。管理对话历史是关键：

```
class Conversation:
    """管理多轮对话历史"""

    def __init__(self, system_prompt: str = ""):
        self.messages: list[dict] = []
        self.system_prompt = system_prompt

    def add_user_message(self, content: str):
        self.messages.append({"role": "user", "content": content})

    def add_assistant_message(self, content):
        self.messages.append({"role": "assistant", "content": content})

    def add_tool_results(self, results: list[dict]):
        self.messages.append({"role": "user", "content": results})

    def get_api_params(self) -> dict:
        params = {"messages": self.messages}
        if self.system_prompt:
            params["system"] = self.system_prompt
        return params
```

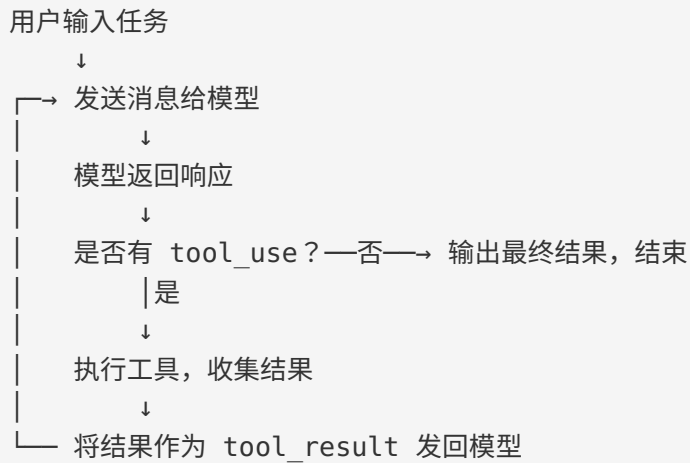
关键概念：在 Claude API 中，对话历史是严格的 user/assistant 交替。工具调用结果（`tool_result`）属于 `user` 角色的消息。每次 API 调用都需要发送完整的对话历史——API 本身是无状态的。

8.2 最小代理实现

现在我们将以上概念整合成一个完整的编程代理。目标：**用约 100 行代码实现一个能读取文件、编辑文件、运行命令的代理。**

8.2.1 代理循环

编程代理的核心是一个循环（agentic loop）：



这个循环会持续运行，直到模型认为任务完成（返回纯文本，不再调用工具）或者达到最大轮次。

8.2.2 定义工具

我们的最小代理需要三个工具：

```
TOOLS = [
    {
        "name": "read_file",
        "description": "读取指定路径的文件内容。用于查看代码、配置文件等。",
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {
                    "type": "string",
                    "description": "要读取的文件路径（相对于项目根目录）"
                }
            }
        },
        "required": ["path"]
    },
    {
        "name": "edit_file",
        "description": "编辑文件。如果文件不存在则创建。用于修改代码、修复 bug 等。",
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {
```

```

        "type": "string",
        "description": "要编辑的文件路径"
    },
    "content": {
        "type": "string",
        "description": "文件的完整新内容"
    }
},
"required": ["path", "content"]
}
},
{
    "name": "run_command",
    "description": "在 shell 中执行命令。用于运行测试、安装依赖、查看目录结构等。",
    "input_schema": {
        "type": "object",
        "properties": {
            "command": {
                "type": "string",
                "description": "要执行的 shell 命令"
            }
        },
        "required": ["command"]
    }
}
}
]

```

8.2.3 完整实现

以下是一个完整、可运行的最小编程代理：

```

#!/usr/bin/env python3
"""
minimal_agent.py - 一个约 100 行的最小编程代理

用法: python minimal_agent.py "修复 calculator.py 中除以零的 bug"
"""

import sys
import os
import subprocess
import anthropic

# — 工具定义 —————

TOOLS = [
    {
        "name": "read_file",

```



```

        "description": "读取指定路径的文件内容",
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {"type": "string", "description": "文件路径"}
            },
            "required": ["path"]
        }
    },
    {
        "name": "edit_file",
        "description": "写入文件（完整覆盖）。如果文件不存在则创建。",
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {"type": "string", "description": "文件路径"},
                "content": {"type": "string", "description": "文件的完整新内
容"}
            },
            "required": ["path", "content"]
        }
    },
    {
        "name": "run_command",
        "description": "在 shell 中执行命令并返回输出",
        "input_schema": {
            "type": "object",
            "properties": {
                "command": {"type": "string", "description": "Shell 命令"}
            },
            "required": ["command"]
        }
    }
]

```

— 工具执行 —————

```

def execute_tool(name: str, args: dict) -> str:
    if name == "read_file":
        try:
            with open(args["path"], "r") as f:
                return f.read()
        except FileNotFoundError:
            return f"错误: 文件 '{args['path']}' 不存在"

    elif name == "edit_file":
        os.makedirs(os.path.dirname(args["path"]) or ".", exist_ok=True)
        with open(args["path"], "w") as f:
            f.write(args["content"])
        return f"已写入 {args['path']} ({len(args['content'])} 字符)"

```

```

elif name == "run_command":
    result = subprocess.run(
        args["command"], shell=True,
        capture_output=True, text=True, timeout=30
    )
    output = result.stdout + result.stderr
    return output[:5000] or "(无输出)" # 截断过长输出

return f"未知工具: {name}"

```

— System Prompt —————

SYSTEM_PROMPT = """你是一个编程代理。你可以读取文件、编辑文件、运行命令来完成编程任务。

工作流程：

1. 先理解任务，阅读相关文件
2. 制定修改方案
3. 编辑文件进行修改
4. 运行测试验证修改是否正确

重要规则：

- 在修改代码之前，先读取文件了解现有代码
- 修改后要运行测试或验证命令确认结果
- 如果测试失败，分析错误并修复"""

— 代理主循环 —————

```

def run_agent(task: str, max_turns: int = 10):
    client = anthropic.Anthropic()
    messages = [{"role": "user", "content": task}]

    for turn in range(max_turns):
        print(f"\n{'='*50}")
        print(f"Turn {turn + 1}/{max_turns}")
        print(f"{'='*50}")

        response = client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=4096,
            system=SYSTEM_PROMPT,
            tools=TTOOLS,
            messages=messages,
        )

        # 将 assistant 回复加入历史
        messages.append({"role": "assistant", "content": response.content})

        # 处理响应内容
        tool_results = []

```

```

    for block in response.content:
        if block.type == "text":
            print(f"\n代理: {block.text}")
        elif block.type == "tool_use":
            print(f"\n[调用 {block.name}] {block.input}")
            result = execute_tool(block.name, block.input)
            print(f"[结果] {result[:200]}...")
            tool_results.append({
                "type": "tool_result",
                "tool_use_id": block.id,
                "content": result
            })

    # 如果没有工具调用, 代理完成了任务
    if not tool_results:
        print("\n✓ 代理已完成任务")
        return

    # 将工具结果发回模型
    messages.append({"role": "user", "content": tool_results})

print("\n△ 达到最大轮次限制")

# — 入口 —
if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("用法: python minimal_agent.py <任务描述>")
        sys.exit(1)
    run_agent(sys.argv[1])

```

8.2.4 Demo: 修复一个 Bug

创建一个有 bug 的文件来测试我们的代理:

```

# calculator.py - 一个有 bug 的计算器
def divide(a, b):
    return a / b # 没有处理除以零的情况

def calculate(expression):
    parts = expression.split()
    a, op, b = float(parts[0]), parts[1], float(parts[2])
    if op == '+': return a + b
    if op == '-': return a - b
    if op == '*': return a * b
    if op == '/': return divide(a, b)

```

```
# test_calculator.py
from calculator import divide, calculate

def test_divide_normal():
    assert divide(10, 2) == 5.0

def test_divide_by_zero():
    try:
        divide(10, 0)
        assert False, "应该抛出异常"
    except ZeroDivisionError:
        pass # 期望行为：抛出明确的错误

def test_calculate():
    assert calculate("10 + 5") == 15.0
    assert calculate("10 / 2") == 5.0
```

运行代理：

```
python minimal_agent.py "运行 pytest test_calculator.py, 如果有测试失败, 修复
calculator.py 中的 bug"
```

代理会：1. 运行 `pytest test_calculator.py`，发现 `test_divide_by_zero` 失败 2. 读取 `calculator.py`，分析问题 3. 修改 `divide` 函数，添加除零检查 4. 重新运行测试，确认全部通过

思考：这个 100 行代理已经能完成简单任务了。但它有明显的局限性：无法搜索代码库、没有错误恢复机制、不会制定计划。接下来的章节将逐步解决这些问题。

8.3 加入 Grounding

我们的最小代理只能操作已知路径的文件。在真实项目中，代理需要在代码库中搜索和定位信息的能力——这就是 grounding。

我们提供三种方案，你可以根据需要选择其一或组合使用。

方案 A：基于 Grep 的搜索（最简单）

最直接的方式——给代理一个代码搜索工具：

```

# 新增工具定义
GREG_TOOL = {
    "name": "search_code",
    "description": "在代码库中搜索匹配模式的内容。返回匹配的文件名和行号。",
    "input_schema": {
        "type": "object",
        "properties": {
            "pattern": {
                "type": "string",
                "description": "搜索模式（支持正则表达式）"
            },
            "file_pattern": {
                "type": "string",
                "description": "文件匹配模式，例如 '*.py'",
                "default": "*"
            }
        },
        "required": ["pattern"]
    }
}

def search_code(pattern: str, file_pattern: str = "*") -> str:
    """使用 ripgrep 或 grep 搜索代码"""
    # 优先使用 ripgrep (更快)
    try:
        result = subprocess.run(
            ["rg", "--line-number", "--type-add",
             f"custom:*.{file_pattern.lstrip('*')} " if file_pattern !=
            """ else """
            pattern, "."],
            capture_output=True, text=True, timeout=10
        )
        if result.stdout:
            # 限制输出行数，避免过长
            lines = result.stdout.strip().split("\n")
            if len(lines) > 50:
                return "\n".join(lines[:50]) + f"\n... (共 {len(lines)} 个匹
            配)"
            return result.stdout
        return "未找到匹配结果"
    except FileNotFoundError:
        # 如果没有 ripgrep，降级到 grep
        result = subprocess.run(
            ["grep", "-rn", "--include", file_pattern, pattern, "."],
            capture_output=True, text=True, timeout=10
        )
        return result.stdout or "未找到匹配结果"

```

优势：实现简单，对大多数任务够用。模型能很好地利用 grep 进行代码导航。

方案 B: 基于 AST 的代码导航

使用 tree-sitter 解析代码结构，提供更精确的导航能力：

```
pip install tree-sitter tree-sitter-python
```

```
import tree_sitter_python as tspython
from tree_sitter import Language, Parser

PY_LANGUAGE = Language(tspython.language())
parser = Parser(PY_LANGUAGE)

AST_TOOL = {
    "name": "find_symbols",
    "description": "查找代码中的函数、类、方法定义。比文本搜索更精确。",
    "input_schema": {
        "type": "object",
        "properties": {
            "path": {
                "type": "string",
                "description": "要分析的文件路径"
            },
            "symbol_type": {
                "type": "string",
                "enum": ["function", "class", "method", "all"],
                "description": "要查找的符号类型"
            },
            "name": {
                "type": "string",
                "description": "可选：按名称过滤",
                "default": ""
            }
        },
        "required": ["path"]
    }
}

def find_symbols(path: str, symbol_type: str = "all", name: str = "") -> str:
    """使用 tree-sitter 查找代码符号"""
    with open(path, "rb") as f:
        source = f.read()

    tree = parser.parse(source)
    results = []

    # 定义要查找的节点类型
    type_map = {
```

```

        "function": ["function_definition"],
        "class": ["class_definition"],
        "method": ["function_definition"], # 在类内部的 function
        "all": ["function_definition", "class_definition"]
    }
    target_types = type_map.get(symbol_type, type_map["all"])

    def visit(node, depth=0):
        if node.type in target_types:
            # 获取符号名称
            for child in node.children:
                if child.type == "identifier":
                    sym_name =
                    source[child.start_byte:child.end_byte].decode()
                    if not name or name in sym_name:
                        line = node.start_point[0] + 1
                        results.append(f" 第 {line} 行: {node.type}
'{sym_name}'")
                    break
            for child in node.children:
                visit(child, depth + 1)

    visit(tree.root_node)
    if results:
        return f"{path} 中找到的符号:\n" + "\n".join(results)
    return f"{path} 中未找到匹配的符号"

```

优势：理解代码结构，能精确定位函数/类定义，不受注释或字符串干扰。

方案 C：基于 RAG 的语义检索

对于大型代码库，可以使用 embedding 实现语义搜索：

```
pip install openai faiss-cpu
```

```

import hashlib
import json
import numpy as np

# 需要额外安装: pip install openai faiss-cpu
import faiss
from openai import OpenAI

RAG_TOOL = {
    "name": "semantic_search",
    "description": "在代码库中进行语义搜索。适合用自然语言描述查找相关代码。",
    "input_schema": {
        "type": "object",

```

```

        "properties": {
            "query": {
                "type": "string",
                "description": "自然语言搜索查询，例如 '处理用户认证的函数'"
            },
            "top_k": {
                "type": "integer",
                "description": "返回最相关的结果数量",
                "default": 5
            }
        },
        "required": ["query"]
    }
}

```

```

class CodeIndex:

```

```

    """代码库的向量索引"""

```

```

    def __init__(self, project_path: str):
        self.project_path = project_path
        self.openai_client = OpenAI()
        self.chunks: list[dict] = [] # {path, start_line, content}
        self.index = None

```

```

    def _get_embedding(self, text: str) -> list[float]:
        response = self.openai_client.embeddings.create(
            model="text-embedding-3-small",
            input=text[:8000] # 截断过长文本
        )
        return response.data[0].embedding

```

```

    def build_index(self):
        """扫描项目文件，构建向量索引"""
        self.chunks = []

```

```

        for root, dirs, files in os.walk(self.project_path):
            # 跳过隐藏目录和常见的非代码目录
            dirs[:] = [d for d in dirs
                        if not d.startswith('.') and d not in
                        ('node_modules', '__pycache__', 'venv', '.git')]

```

```

        for file in files:
            if not file.endswith(('py', 'js', 'ts', 'go', 'rs')):
                continue
            filepath = os.path.join(root, file)
            try:
                with open(filepath, 'r') as f:
                    content = f.read()
            except (UnicodeDecodeError, PermissionError):
                continue

```



```

        # 按函数/类定义分块 (简化版：按空行分段)
        lines = content.split('\n')
        chunk_size = 50 # 每块约 50 行
        for i in range(0, len(lines), chunk_size):
            chunk_lines = lines[i:i + chunk_size]
            chunk_content = '\n'.join(chunk_lines)
            if chunk_content.strip():
                self.chunks.append({
                    "path": os.path.relpath(filepath,
self.project_path),
                    "start_line": i + 1,
                    "content": chunk_content
                })

        if not self.chunks:
            return

        # 批量生成 embeddings
        embeddings = []
        batch_size = 100
        for i in range(0, len(self.chunks), batch_size):
            batch = self.chunks[i:i + batch_size]
            batch_texts = [c["content"] for c in batch]
            response = self.openai_client.embeddings.create(
                model="text-embedding-3-small",
                input=batch_texts
            )
            embeddings.extend([d.embedding for d in response.data])

        # 构建 FAISS 索引
        dim = len(embeddings[0])
        self.index = faiss.IndexFlatIP(dim) # 内积相似度
        vectors = np.array(embeddings, dtype=np.float32)
        faiss.normalize_L2(vectors) # 归一化后内积 = 余弦相似度
        self.index.add(vectors)

        print(f"索引构建完成: {len(self.chunks)} 个代码片段")

    def search(self, query: str, top_k: int = 5) -> str:
        if self.index is None:
            return "错误: 索引尚未构建, 请先调用 build_index()"

        query_embedding = np.array(
            [self._get_embedding(query)], dtype=np.float32
        )
        faiss.normalize_L2(query_embedding)
        scores, indices = self.index.search(query_embedding, top_k)

        results = []
        for score, idx in zip(scores[0], indices[0]):
            if idx < 0:

```

```

        continue
    chunk = self.chunks[idx]
    results.append(
        f"--- {chunk['path']}:{chunk['start_line']} "
        f"(相关度: {score:.3f}) ---\n{chunk['content']}"
    )

    return "\n\n".join(results) if results else "未找到相关代码"

```

优势：支持自然语言查询（如“处理用户登录的代码”），适合大型代码库。

8.3.4 将搜索工具集成到代理

选择一种方案后，将其加入代理的工具列表即可：

```

# 在 minimal_agent.py 的基础上扩展
TOOLS = [
    # ... 原有的 read_file, edit_file, run_command ...
    GREP_TOOL, # 或 AST_TOOL 或 RAG_TOOL
]

def execute_tool(name: str, args: dict) -> str:
    # ... 原有的工具处理 ...
    if name == "search_code":
        return search_code(args["pattern"], args.get("file_pattern", ""))
    # 如果用 AST 方案：
    # if name == "find_symbols":
    #     return find_symbols(args["path"], args.get("symbol_type", "all"),
    #                        args.get("name", ""))
    return f"未知工具: {name}"

```

现在代理可以在不知道文件路径的情况下搜索代码库了。例如：

```
python minimal_agent.py "在项目中找到处理用户认证的代码，然后添加密码长度校验"
```

代理会先搜索“auth”或“login”相关代码，找到对应文件，读取后进行修改。

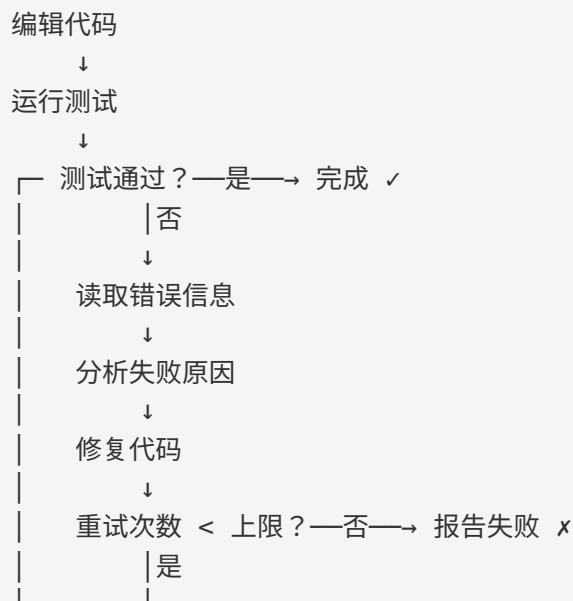
实践建议：从方案 A（grep）开始，它的实现最简单，效果已经很好。大多数商业编程代理（包括 Claude Code）的核心搜索能力也是基于 grep/ripgrep。只在确实需要时再升级到 AST 或 RAG。

8.4 加入 Feedback

代理写的代码不一定第一次就对。通过**反馈循环（feedback loop）**，代理可以自动检测错误并修复，显著提高成功率。

8.4.1 测试驱动的反反馈循环

核心思路：编辑 → 测试 → 如果失败，读取错误信息 → 修复 → 重新测试。



实际上，我们不需要显式实现这个循环——如果 system prompt 中包含了“修改后运行测试，失败就修复”的指令，代理循环本身就会执行这个流程。但我们可以通过改进 system prompt 和增加约束来强化这个行为。

8.4.2 强化后的 System Prompt

```
SYSTEM_PROMPT_WITH_FEEDBACK = """你是一个编程代理。你可以读取文件、编辑文件、运行命令来完成编程任务。
```

```
## 工作流程（必须严格遵循）
```

1. ****理解阶段****：阅读相关代码文件，理解上下文
2. ****计划阶段****：在编辑之前，先说明你打算如何修改
3. ****编辑阶段****：进行代码修改
4. ****验证阶段****：运行测试或其他验证命令
5. ****修复阶段****：如果验证失败，分析错误并修复。最多重试 3 次。

```
## 重要规则
```

- 每次编辑后必须运行测试验证
- 如果测试失败，仔细阅读错误信息，不要盲目修改
- 如果 3 次修复后仍然失败，停下来说明问题所在
- 不要删除或跳过失败的测试
- 保持修改最小化，只改必要的部分"""

8.4.3 程序化的重试机制

虽然好的 prompt 通常足够，但你也可以在代码层面强制重试：

```
def run_agent_with_feedback(task: str, test_command: str,
                             max_retries: int = 3):
    """带有强制测试验证的代理运行器"""
    client = anthropic.Anthropic()

    # 第一步：让代理完成任务
    enhanced_task = f"""{task}

完成修改后，请运行以下测试命令验证：
{test_command}

如果测试失败，请分析错误并修复代码，然后重新运行测试。"""

    messages = [{"role": "user", "content": enhanced_task}]
    retry_count = 0

    while retry_count < max_retries:
        # 运行代理直到它完成（不再调用工具）
        response = run_until_done(client, messages)

        # 代理认为它完成了，但让我们独立验证
        test_result = subprocess.run(
            test_command, shell=True,
            capture_output=True, text=True, timeout=60
        )

        if test_result.returncode == 0:
            print(f"\n✓ 测试通过！（重试了 {retry_count} 次）")
            return True

        # 测试失败，将错误反馈给代理
        retry_count += 1
        error_msg = test_result.stdout + test_result.stderr
        print(f"\n✗ 测试失败（第 {retry_count}/{max_retries} 次重试）")

        messages.append({
            "role": "user",
```

```
"content": f"""测试仍然失败。这是第 {retry_count} 次重试。
```

错误输出：

```
{error_msg[:3000]}
```

```
请分析失败原因并修复。注意不要重复之前的错误。"""
    })
```

```
print(f"\nx 达到最大重试次数 ({max_retries}), 任务失败")
return False
```

```
def run_until_done(client, messages):
    """运行代理直到它不再调用工具"""
    while True:
        response = client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=4096,
            system=SYSTEM_PROMPT_WITH_FEEDBACK,
            tools=TOOLS,
            messages=messages,
        )

        messages.append({"role": "assistant", "content": response.content})

        tool_results = []
        for block in response.content:
            if block.type == "text":
                print(f"代理: {block.text}")
            elif block.type == "tool_use":
                result = execute_tool(block.name, block.input)
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": result
                })

        if not tool_results:
            return response

        messages.append({"role": "user", "content": tool_results})
```

8.4.4 自我评审 (Self-Review)

在提交最终结果前，让模型评审自己的修改：

```
def self_review(client, original_code: str, modified_code: str,
                task: str) -> dict:
    """让模型评审自己的修改"""
    review_prompt = f"""请评审以下代码修改。

## 任务
{task}

## 原始代码
```

```
{original_code}
```

```
## 修改后的代码
```

```
{modified_code}
```

请从以下几个方面评审：

1. 修改是否正确解决了问题？
2. 是否引入了新的 bug？
3. 代码风格是否与原代码一致？
4. 是否有遗漏的边界情况？

以 JSON 格式回答：

```
{{"approved": true/false, "issues": ["问题1", "问题2"], "suggestions": ["建议1"]}}
```

```
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=[{"role": "user", "content": review_prompt}]
)
```

```
# 解析 JSON 回复
import re
text = response.content[0].text
json_match = re.search(r'\{.*\}', text, re.DOTALL)
if json_match:
    return json.loads(json_match.group())
return {"approved": True, "issues": [], "suggestions": []}
```

8.4.5 反馈循环的效果

根据 SWE-bench 等基准测试的数据，反馈循环对代理成功率的提升非常显著：

策略	相对成功率
无反馈（一次性生成）	基准
编辑后运行测试 + 修复	+30-50%
加上自我评审	+10-15%
加上错误分类和针对性修复	+5-10%

关键洞察： 反馈循环是编程代理中投入产出比最高的功能之一。即使是简单的“编辑→测试→修复”循环，也能带来大幅提升。这也是为什么所有主流编程代理都将测试运行作为核心能力。

8.5 加入 Planning

对于复杂任务（跨多个文件、涉及多个步骤），代理需要先制定计划再执行。没有计划的代理容易在复杂任务中迷失方向。

8.5.1 两阶段执行

Plan-then-Act 架构将任务分为两个阶段：



8.5.2 实现 Planning 阶段

```
PLANNING_SYSTEM_PROMPT = """你是一个编程代理的规划模块。你的任务是分析编程任务并制定详细的执行计划。
```

你可以使用工具来阅读代码、搜索文件，以便了解代码库结构。

最终输出一个 JSON 格式的计划：

```
{
  "analysis": "任务分析",
  "steps": [
    {
      "id": 1,
      "description": "步骤描述",
      "files": ["需要修改的文件"],
      "verification": "验证方法"
    }
  ],
  "risks": ["潜在风险"],
  "estimated_complexity": "low/medium/high"
}
```

不要执行任何修改，只负责分析和规划。"""

```
EXECUTION_SYSTEM_PROMPT = """你是一个编程代理。请按照给定的计划逐步执行任务。
```

规则：

- 严格按照计划的步骤顺序执行
- 每完成一步，执行该步骤的验证
- 如果某步失败，尝试修复；如果无法修复，说明情况
- 不要跳过步骤
- 不要做计划之外的修改"""

```
def plan_and_execute(task: str, require_approval: bool = False):
    """两阶段执行：先规划，后执行"""
    client = anthropic.Anthropic()

    # — 阶段一：Planning —————
    print("=" * 50)
    print("阶段一：Planning")
    print("=" * 50)

    # 规划阶段使用只读工具（不包含 edit_file）
    readonly_tools = [t for t in T00LS if t["name"] != "edit_file"]

    plan_messages = [{"role": "user", "content": f"""请为以下任务制定执行计划：

{task}"""
```


先用工具了解代码库结构和相关文件，然后输出计划。"""}]

```
# 运行规划代理
plan = run_until_done_with_tools(
    client, plan_messages,
    system=PLANNING_SYSTEM_PROMPT,
    tools=readonly_tools
)

# 提取计划文本
plan_text = ""
for block in plan.content:
    if block.type == "text":
        plan_text += block.text

print(f"\n计划:\n{plan_text}")

# — 可选：人工审批 —————
if require_approval:
    print("\n" + "=" * 50)
    user_input = input("是否批准这个计划？(yes/no/修改意见): ").strip()
    if user_input.lower() == "no":
        print("计划被拒绝，终止执行。")
        return
    elif user_input.lower() != "yes":
        # 用户给出了修改意见
        plan_text += f"\n\n用户反馈: {user_input}"

# — 阶段二：Execution —————
print("\n" + "=" * 50)
print("阶段二：Execution")
print("=" * 50)

exec_messages = [{"role": "user", "content": f"""请按照以下计划执行任务：
```

原始任务

{task}

执行计划

{plan_text}

请逐步执行，每步完成后进行验证。"""}]

```
run_until_done_with_tools(
    client, exec_messages,
    system=EXECUTION_SYSTEM_PROMPT,
    tools=TOOLS
)
```

```

print("\n✓ 执行完成")

def run_until_done_with_tools(client, messages, system, tools):
    """通用的运行直到完成的循环"""
    while True:
        response = client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=4096,
            system=system,
            tools=tools,
            messages=messages,
        )
        messages.append({"role": "assistant", "content": response.content})

        tool_results = []
        for block in response.content:
            if block.type == "text":
                print(f" {block.text[:200]}")
            elif block.type == "tool_use":
                print(f" [工具: {block.name}]")
                result = execute_tool(block.name, block.input)
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": result
                })

            if not tool_results:
                return response

        messages.append({"role": "user", "content": tool_results})

```

8.5.3 动态重新规划

有时执行过程中会遇到计划外的情况。代理需要能够重新规划：

```
ADAPTIVE_SYSTEM_PROMPT = """你是一个编程代理。你有一个初始计划，但可以根据情况调整。
```

当前计划：
{plan}

执行规则：

- 按计划执行，但如果发现计划有误，可以调整
- 如果需要偏离计划，先说明原因
- 每步完成后检查是否需要调整后续步骤
- 保持修改最小化"""

```

def adaptive_execute(task: str, plan: str):
    """支持动态调整的执行"""
    client = anthropic.Anthropic()
    system = ADAPTIVE_SYSTEM_PROMPT.format(plan=plan)

    messages = [{"role": "user", "content": f"开始执行任务: {task}"}]

    step_count = 0
    max_steps = 20

    while step_count < max_steps:
        response = client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=4096,
            system=system,
            tools=TOOLS,
            messages=messages,
        )

        messages.append({"role": "assistant", "content": response.content})

        tool_results = []
        for block in response.content:
            if block.type == "text":
                # 检测是否提到了重新规划
                if "重新规划" in block.text or "调整计划" in block.text:
                    print(f"🔄 代理正在重新规划...")
                    print(f"    {block.text[:300]}")
            elif block.type == "tool_use":
                step_count += 1
                result = execute_tool(block.name, block.input)
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": result
                })

        if not tool_results:
            break

        messages.append({"role": "user", "content": tool_results})

```

8.5.4 Planning 的利弊

Planning 有帮助的场景： - 跨多个文件的修改（如重构一个接口） - 需要按特定顺序执行的步骤 - 复杂的功能开发（多个组件协调） - 用户需要在执行前了解代理的意图

Planning 可能有害的场景： - 简单、独立的 bug 修复 - 计划生成本身消耗大量 token - 计划过于详细导致执行僵化，无法灵活应对变化 - 任务本身就不确定，需要探索式解决

设计建议：可以让代理自行决定是否需要规划。对简单任务直接执行，对复杂任务启动规划。Claude Code 采用的就是这种混合策略——模型根据任务复杂度自行决定是否输出显式计划。

8.6 企业级考量

在把编程代理从原型推向生产环境时，需要考虑以下关键问题。

8.6.1 数据流安全

编程代理处理的是你的源代码——这可能是公司最核心的资产。

核心问题：哪些数据离开了你的环境？

你的代码 → API 请求 → 模型提供商 → API 响应 → 代理

↑

这里发送了什么？

缓解措施：

```
class SecureAgent:
    """安全感知的代理封装"""

    # 敏感文件模式
    SENSITIVE_PATTERNS = [
        ".env", ".env.*", "*.pem", "*.key",
        "*credentials*", "*secret*", "*token*",
        ".git/config", # 可能包含认证信息
    ]

    def __init__(self):
        self.sent_files: list[str] = [] # 审计追踪

    def read_file_safe(self, path: str) -> str:
        """读取文件前检查是否为敏感文件"""
        import fnmatch
        for pattern in self.SENSITIVE_PATTERNS:
            if fnmatch.fnmatch(os.path.basename(path), pattern):
                return f"[已阻止] 文件 {path} 匹配敏感模式 '{pattern}', 不会发送到 API"
```

```

        content = open(path).read()
        self.sent_files.append(path) # 记录
        return content

def redact_secrets(self, text: str) -> str:
    """从文本中移除可能的密钥"""
    import re
    # 移除常见的 API Key 模式
    patterns = [
        (r'(sk-[a-zA-Z0-9]{20,})', '[REDACTED_API_KEY]'),
        (r'(ghp_[a-zA-Z0-9]{36})', '[REDACTED_GITHUB_TOKEN]'),
        (r'(password\s*=\s*["\'])(^"\'|'+(["\']))',
         r'\1[REDACTED]\2'),
    ]
    for pattern, replacement in patterns:
        text = re.sub(pattern, replacement, text)
    return text

```

8.6.2 本地/自托管模型

对于严格的隐私要求，可以使用本地模型：

```

# 使用 Ollama 运行本地模型
# 安装: curl -fsSL https://ollama.com/install.sh | sh
# 拉取模型: ollama pull qwen2.5-coder:32b

from openai import OpenAI # Ollama 兼容 OpenAI API

def create_local_client():
    """创建连接本地 Ollama 的客户端"""
    return OpenAI(
        base_url="http://localhost:11434/v1",
        api_key="ollama" # Ollama 不需要真正的 key
    )

# 使用方式与 OpenAI API 相同
local_client = create_local_client()
response = local_client.chat.completions.create(
    model="qwen2.5-coder:32b",
    messages=[{"role": "user", "content": "Hello"}],
    # 注意：本地模型对 tool use 的支持可能有限
)

```

模型选择矩阵：

需求	推荐方案
最高质量	Claude claude-sonnet-4-20250514 / Opus (API)
数据不出境	本地部署 Qwen-Coder / DeepSeek-Coder
低延迟	Claude Haiku / 本地小模型
离线环境	Ollama + 量化模型

8.6.3 成本控制

编程代理可能消耗大量 token。实用的成本控制策略：

```
class CostController:
    """Token 用量和成本控制"""

    # 近似定价 (USD per 1M tokens, Claude Sonnet)
    INPUT_PRICE = 3.0 # 输入
    OUTPUT_PRICE = 15.0 # 输出

    def __init__(self, budget_usd: float = 1.0):
        self.budget = budget_usd
        self.total_input_tokens = 0
        self.total_output_tokens = 0

    @property
    def cost_usd(self) -> float:
        return (
            self.total_input_tokens * self.INPUT_PRICE / 1_000_000
            + self.total_output_tokens * self.OUTPUT_PRICE / 1_000_000
        )

    def track(self, response) -> bool:
        """记录 token 使用量, 如果超预算返回 False"""
        self.total_input_tokens += response.usage.input_tokens
        self.total_output_tokens += response.usage.output_tokens

        if self.cost_usd > self.budget:
            print(f"⚠ 预算超限: ${self.cost_usd:.4f} / ${self.budget:.2f}")
            return False
        return True

    def summary(self) -> str:
        return (
            f"Token 使用: {self.total_input_tokens:,} 输入 + "
            f"{self.total_output_tokens:,} 输出\n"
            f"预估成本: ${self.cost_usd:.4f}"
        )
```

```
# 使用
cost = CostController(budget_usd=0.50)

response = client.messages.create(...)
if not cost.track(response):
    print("预算耗尽, 停止代理")
    print(cost.summary())
```

其他成本优化技巧：

- **Prompt caching：**Anthropic API 支持 prompt caching，对长 system prompt 和多轮对话可节省大量输入 token 费用
- **模型路由：**简单任务用 Haiku，复杂任务用 Sonnet/Opus
- **上下文截断：**定期压缩对话历史，避免无限增长
- **缓存工具结果：**对同一文件的重复读取进行本地缓存

8.6.4 审计日志

在企业环境中，记录代理的每一步操作是合规要求：

```
import json
import time
from datetime import datetime

class AuditLogger:
    """代理操作审计日志"""

    def __init__(self, log_file: str = "agent_audit.jsonl"):
        self.log_file = log_file

    def log(self, event_type: str, details: dict):
        entry = {
            "timestamp": datetime.utcnow().isoformat(),
            "event_type": event_type,
            **details
        }
        with open(self.log_file, "a") as f:
            f.write(json.dumps(entry, ensure_ascii=False) + "\n")

    def log_tool_call(self, tool_name: str, args: dict, result: str):
        self.log("tool_call", {
            "tool": tool_name,
            "arguments": args,
            "result_preview": result[:500], # 截断长结果
            "result_length": len(result)
        })

    def log_model_call(self, input_tokens: int, output_tokens: int,
                       model: str):
```

```

        self.log("model_call", {
            "model": model,
            "input_tokens": input_tokens,
            "output_tokens": output_tokens
        })

    def log_file_modification(self, path: str, action: str):
        self.log("file_modification", {
            "path": path,
            "action": action # "create", "edit", "delete"
        })

# 集成到代理中
audit = AuditLogger()

def execute_tool_audited(name: str, args: dict) -> str:
    result = execute_tool(name, args)
    audit.log_tool_call(name, args, result)

    if name == "edit_file":
        audit.log_file_modification(args["path"], "edit")

    return result

```

8.6.5 访问控制

限制代理在生产环境中可以做什么：

```

class Sandbox:
    """代理的沙箱环境"""

    def __init__(self, allowed_dirs: list[str],
                 allowed_commands: list[str],
                 read_only: bool = False):
        self.allowed_dirs = [os.path.abspath(d) for d in allowed_dirs]
        self.allowed_commands = allowed_commands
        self.read_only = read_only

    def check_path(self, path: str) -> bool:
        """检查路径是否在允许范围内"""
        abs_path = os.path.abspath(path)
        return any(abs_path.startswith(d) for d in self.allowed_dirs)

    def check_command(self, command: str) -> bool:
        """检查命令是否在白名单中"""
        cmd_name = command.split()[0] if command else ""
        return cmd_name in self.allowed_commands

```



```

def execute_tool(self, name: str, args: dict) -> str:
    if name in ("read_file", "edit_file"):
        if not self.check_path(args["path"]):
            return f"[权限拒绝] 路径 {args['path']} 不在允许范围内"
        if name == "edit_file" and self.read_only:
            return "[权限拒绝] 当前为只读模式"

    if name == "run_command":
        if not self.check_command(args["command"]):
            return f"[权限拒绝] 命令 '{args['command']}' 不在白名单中"

    return execute_tool(name, args)

# 使用
sandbox = Sandbox(
    allowed_dirs=["./src", "./tests"],
    allowed_commands=["pytest", "python", "npm", "ls", "cat", "grep"],
    read_only=False
)

```

8.6.6 CI/CD 集成

将编程代理嵌入自动化流水线：

```

# .github/workflows/agent-fix.yml
# 当测试失败时，自动运行代理尝试修复

name: Agent Auto-Fix
on:
  workflow_run:
    workflows: ["CI Tests"]
    types: [completed]

jobs:
  auto-fix:
    if: ${ github.event.workflow_run.conclusion == 'failure' }
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Get test failure logs
        run: |
          # 获取失败的测试日志
          gh run view ${ github.event.workflow_run.id } --log-failed > /
          tmp/failures.log
        env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }

      - name: Run coding agent

```

```
run: |
    python agent.py "根据以下测试失败日志修复代码: $(cat /tmp/
failures.log)"
env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }

- name: Create PR with fix
  run: |
    git checkout -b auto-fix/${ github.run_id }
    git add -A
    git commit -m "fix: auto-fix test failures (agent-generated)"
    gh pr create --title "Auto-fix: test failures" \
        --body "This PR was generated by the coding agent."
env:
    GH_TOKEN: ${ secrets.GITHUB_TOKEN }
```

重要提醒：在 CI/CD 中使用代理时，务必设置严格的沙箱、预算限制和人工审批环节。自动化 + AI 代理 = 强大但需要控制。

实操环节（结业项目）

项目目标

从零构建一个面向特定领域的编程代理，综合运用本课程所学的全部概念。

项目要求

你的代理必须包含以下能力：

- 核心工具**（至少 3 个）：文件读写、命令执行、代码搜索
- Grounding 机制**：代理能在代码库中搜索和定位信息
- Feedback Loop**：编辑后自动验证，失败时自动修复
- Planning**（可选加分）：对复杂任务先规划后执行
- 安全措施**：至少包含路径检查和敏感文件过滤

建议选题方向

选题 A：Web API 开发代理 - 根据自然语言描述自动生成 RESTful API endpoint - 自动生成对应的测试用例 - 运行测试验证 API 行为 - 支持 FastAPI / Express / Gin 等框架

选题 B：数据管道代理 - 根据数据处理需求自动生成 ETL pipeline - 支持数据验证和质量检查 - 处理失败时自动诊断和修复 - 支持 Pandas / PySpark / dbt 等工具

选题 C：DevOps 自动化代理 - 根据需求生成 Dockerfile / docker-compose.yml - 自动配置 CI/CD pipeline - 检查配置错误并修复 - 支持 GitHub Actions / GitLab CI

选题 D：自由选题 - 任何你感兴趣的领域 - 需要在提案中说明选题理由和技术方案

评估标准

维度	权重	说明
功能完整性	30%	核心功能是否都已实现并可运行
代码质量	20%	架构是否清晰、代码是否整洁
创新性	15%	是否有独特的设计决策或功能
实用性	15%	是否真的能解决实际问题
文档与演示	10%	README 是否清晰，演示是否有说服力
安全考量	10%	是否考虑了安全和隐私问题

交付物

- 1. **代码仓库**：完整、可运行的代理代码
- 2. **README.md**：包含安装说明、使用方法、架构说明
- 3. **演示视频/录屏**（3-5 分钟）：展示代理完成一个真实任务的全过程
- 4. **设计文档**（1-2 页）：说明关键的设计决策及其权衡

时间安排

阶段	时间	内容
选题与提案	第 1 周	确定选题，提交技术方案
核心实现	第 2-3 周	实现基础代理 + 工具 + feedback loop
完善与测试	第 4 周	添加 planning、安全措施，全面测试
文档与演示	第 5 周	撰写文档，录制演示

本模块小结

本模块我们从零开始构建了一个完整的编程代理，经历了以下关键阶段：

1. **API 基础**：掌握了 tool use、streaming、多轮对话等核心 API 能力
2. **最小代理**：用约 100 行代码实现了一个可工作的代理循环
3. **Grounding**：通过代码搜索工具让代理能在大型代码库中导航
4. **Feedback**：通过测试驱动的反饋循环大幅提升了代理的正确率
5. **Planning**：加入规划机制让代理能处理复杂的多步骤任务
6. **企业考量**：讨论了安全、隐私、成本和审计等生产环境问题

关键收获

- **代理 = 循环 + 工具 + 模型**。核心架构出奇地简单。
 - **工具设计决定能力上限**。模型再强大，没有合适的工具也无法完成任务。
 - **反馈循环是最高 ROI 的投资**。让代理能自我验证和修复，比升级模型更有效。
 - **复杂性应该逐步引入**。从最小可用开始，按需添加 grounding、planning 等能力。
 - **安全不是可选项**。代理能执行代码，就必须有安全边界。
-

思考题

1. **工具设计**：如果你只能给代理 3 个工具，你会选择哪 3 个？为什么？如果增加到 5 个呢？增加的两个工具分别解决什么问题？
2. **Edit 粒度**：我们的 `edit_file` 工具是整文件覆盖。这有什么问题？你能设计一个更好的编辑工具吗？（提示：考虑 diff-based editing、search-and-replace 等方案）
3. **上下文窗口管理**：当对话历史超过模型的上下文窗口时，应该如何处理？有哪些压缩或摘要策略？各有什么优缺点？
4. **多代理协作**：如果能让多个代理协同完成一个大任务（例如一个代理写前端，一个写后端），需要解决哪些协调问题？你会如何设计通信协议？
5. **评估难题**：如何衡量一个编程代理的“好坏”？SWE-bench 类基准测试有什么局限性？你能设计一个更好的评估方案吗？
6. **安全边界**：如果代理运行在一个包含敏感数据的生产环境中，你会设计怎样的安全架构？如何在能力和安全之间找到平衡？

7. **反思：**回顾本课程所有模块的内容，你认为当前编程代理最大的瓶颈是什么？是模型能力、工具设计、上下文管理，还是其他因素？在接下来的 1-2 年里，你预计会出现怎样的突破？